https://claude.ai/share/d866f430-bc30-440e-96c1-0451ebbf5494

Reading this paper: PyPSA-Spain reference paper:

https://www.sciencedirect.com/science/article/pii/S2211467X25001270 (As in the "PyPSA-Spain An extension of PyPSA-Eur to model the Spanish energy system.pdf" attached), I've got inspired and motivated to do the same for Libya.

So, I need your assistance to build "PyPSA-Libya" for modelling the Libyan Electricity Grid. For more context, I have pasted some info about this for your guidance as attached or uploaded in the "PASTED" document in this query here, which is originally my related notes on Keep Google note Let me know if you want additional or further info or details.

We expect to get the related source and outcomes files. So we can then fork or upload them in GitHub Repo, such as:

1. GitHub Repository Architecture
2. What to Commit vs. What to Connect/Ignore
3. Recommended Git Workflow Follow these four steps to build and publish your project: Step 1: Fork an Existing Base Repo [1] Do not build the core workflow from scratch. Fork PyPSA-Earth (designed for global/regional customization) or PyPSA-Eur to your personal or organization GitHub account. Rename your fork to `pypsa-libya`. [1, 2] Step 2: Configure the Target Country Modify the `config.yaml` file in your repository to target Libya. You will limit the spatial boundary using ISO country codes: yaml

countries: ["LY"]

Use code with caution. Commit this configuration change directly to your repo. Step 3: Connect External Storage for Large Files Run your workflow using a tool like Snakemake. The pipeline will download massive weather data and generate large outcome network files locally. [1, 2, 3]

- For personal use: Leave these files in your local `resources/` and `results/` folders (safe behind your `.gitignore`).
- For team sharing: Connect an external cloud storage provider, such as Zenodo, OSF (Open Science Framework), or an AWS S3 bucket, to host and share the large `.nc` netCDF files.

Step 4: Share Code and Data Packages

1. `git add scripts/ config.yaml environment.yaml`
2. `git commit -m "Configure PyPSA for Libyan electricity grid topology"`
3. `git push origin main`
4. In your `README.md`, provide a direct hyperlink to the external cloud storage platform where users can download the pre-calculated raw inputs and model results.

Finally: Integrating a LaTeX-based dissemination step directly into the PyPSA-Libya repository ensures full reproducibility of results, creating a "research-as-code" workflow. Based on the structure of the PyPSA-Spain reference paper, setting up a /paper folder with elsarticle.cls and automating figure exports enables seamless documentation of the methodology and results. Read the structural blueprint for this approach in the PyPSA-Spain reference paper: https://www.sciencedirect.com/science/article/pii/S2211467X25001270

==========

Google Keep Note: (8) Facebook شرح مفصل لأزمة الكهرباء في ليبيا Liby Electric Grid GECOL

—----------

https://www.facebook.com/reel/1062356273394321

دكتور توفيق المنفي

بارك الله في المهندس شرح وافي ويتحدث عن، جانب فني بحث ولا يتكلم علي ادارة شركه الكهرباء واشار ان محطات تم فيهن اجراء، وتوقفن من قبل ادارة الكهرباء.

==========

https://www.facebook.com/reel/2163364630896978

https://www.facebook.com/reel/1587441826283056

Dr-Abdullah Swissi Emhemed

في ظل غياب التقارير الفنية والمعلومة هذا الفيديو يوضح ماحدث يوم امس واليوم بخصوص بلاك آوت #الجنوب.

تتداول ايضا اخبار حدوث فصل دوائر الفجيج اوبارى تزامنا مع فقد التوليد. #بلاك #آوت على مدار يومى لدرجة #جدتى الأمية تعلمت مصطلح Blackout. تنطق فيها 'ناتيف سبيكر'

https://www.researchgate.net/profile/Abdullah-Emhemed

https://ieeexplore.ieee.org/author/37681566500

==========

If you want to build a PyPSA-Libya, a model of Libyan Electricity Grid from scratch, the closest existing models and functional equivalents to the **Libyan grid model** include:

1. PyPSA-Africa (The Direct Geographic Match)
2. PyPSA-Earth (The Global Multi-Sector Base)
3. PyPSA-Spain (The Closest Operational Equivalent)

Because PyPSA-Earth uses automated scripts to extract open data (like OpenStreetMap) for any geographical boundary, you can generate a customized, high-resolution national grid model for Libya or any other MENA

---

https://share.google/aimode/Y0h0D4fyZaBKmXtQk

Yes, there are PyPSA models covering countries in the MENA region, primarily driven by the **PyPSA meets Africa / PyPSA-Africa initiative** and its global counterpart, **PyPSA-Earth**. [1, 2, 3]
Because PyPSA-Earth uses automated scripts to extract open data (like OpenStreetMap) for any geographical boundary, you can generate a customized, high-resolution national grid model for Libya or any other MENA country. [1, 4, 5]
If you do not want to build a model from scratch using PyPSA-Earth, the closest existing models and functional equivalents to the **Libyan grid model** include:

1. PyPSA-Africa (The Direct Geographic Match)

What it is: A fully open-source electricity system model designed for the African continent.
Why it fits Libya: Because Libya is part of the African continent, its transmission lines, demand data, and renewable potentials are already structurally integrated into the master PyPSA-Africa

database. You can isolate the Libyan grid using their regional cutouts and spatial clustering tools. [1, 2, 4, 6]

2. PyPSA-Earth (The Global Multi-Sector Base)

What it is: An extension of the European PyPSA-Eur model scaled to global coverage.
Why it fits Libya: It provides customizable data extraction scripts. You can explicitly clip the coordinates for North Africa / Libya to instantly generate electricity demand time-series, existing power plant data, and medium-to-high-voltage transmission networks. [1, 4, 7]

3. PyPSA-Spain (The Closest Operational Equivalent)

What it is: A specialized, high-resolution national fork of PyPSA-Eur.
Why it fits Libya: While geographically separate, **PyPSA-Spain** offers an ideal template for Libya's unique grid architecture.
The structural similarity: Libya’s power grid operates as a long, narrow coastal transmission backbone (stretching along the Mediterranean from west to east) with massive desert solar potential inland. PyPSA-Spain handles a similar high-solar, highly concentrated coastal demand profile with specific peninsular/isolated network constraints. [8, 9]

Summary of Alternatives

Model
Regional Coverage
Availability for Libya
Best Used For
PyPSA-Africa
All African countries
Included out-of-the-box
Analyzing cross-border MENA connections and local data validation.
PyPSA-Earth
Global
Extractable via script
Building a customized, highly granular model of the Libyan grid.
PyPSA-Spain
Spain
Analogy only
Mirroring code structures for isolated coastal transmission and high solar penetration.

If you want to proceed, I can help you find the **GitHub repository links** for these initiatives or provide the **Python code snippet** required to set up a PyPSA-Earth geographic boundary filter for North Africa. Which would you prefer? [4]

—----------

https://claude.ai/share/d866f430-bc30-440e-96c1-0451ebbf5494

PyPSA-Libya is a sound, well-precedented project — it follows exactly the same pattern as PyPSA-Africa, PyPSA-Ariadne (Germany), PyPSA-GB, and PyPSA-Spain (the paper you attached). I'll set up the actual scaffolding files you can drop into a fork and push.

A few notes before the files:

- **Fork target**: PyPSA-Earth is the right base, not PyPSA-Eur. PyPSA-Eur's data pipeline (ENTSO-E grid, Copernicus land cover for EU) is Europe-only and won't resolve Libya. PyPSA-Earth is explicitly built for OSM-based extraction anywhere in the world (this is also what the Google AI Mode text you pasted concluded). PyPSA-Spain is useful to you only as a *structural template* (nested-model design, national cost overrides, paper structure) — you can't literally fork it, since its data pipeline is hard-wired to Iberia via PyPSA-Eur.

- I'm not able to actually run the Snakemake workflow here (it needs GB-scale weather cutouts, OSM extracts, and a solver license/runtime far beyond this session), so what follows is the real repository scaffolding — config, `.gitignore`, README, and a `/paper` folder with an elsarticle skeleton — rather than a simulated model run.Now the `.gitignore`, README, and the `/paper` skeleton:This is a scaffold you can drop straight onto a fresh fork — not a run of the model itself (that needs GB-scale weather/OSM downloads and a solver, well beyond what I can execute here). It contains:

- **`REPO_STRUCTURE.md`** — the architecture, commit-vs-ignore rules, and full git workflow you asked for, plus why PyPSA-Earth (not PyPSA-Eur) is the right fork target for Libya

- **`config.libya.yaml`** — an overlay config targeting Libya's bounding box, node-count sweep, and its real interconnections (Tunisia, Egypt, optional Italy HVDC)

- **`data_LY/costs/costs_updates.csv`** — placeholder for national cost overrides, same pattern as PyPSA-Spain's `data_ES/`

- **`.gitignore`**, **`README.md`** — with a Zenodo/OSF slot for large files, same as PyPSA-Spain's "Code/Data availability" sections

- **`paper/paper.tex`** + **`Makefile`** — elsarticle skeleton mirroring the Spain paper's section structure, wired to auto-export figures from solved networks rather than hand-placed images

One thing worth flagging honestly: Libya doesn't have Datadis/ESIOS-style open APIs for hourly demand-by-region or capacity-by-region, which PyPSA-Spain leans on heavily (Sections 2.1–2.2). I noted this as a limitation in the README and paper skeleton rather than papering over it — if you have any GECOL data or other sourced figures, that's the piece most worth prioritizing next, since it determines how much of the Q2Q renewable-correction and demand-disaggregation methodology actually transfers.

—----------

https://claude.ai/share/d866f430-bc30-440e-96c1-0451ebbf5494

Here's the practical path from here:

1. Extract it locally

tar -xzf pypsa-libya-scaffold.tar.gz

This gives you the `pypsa-libya-scaffold/` folder with everything from before.

2. Fork the real base repo on GitHub Go to https://github.com/pypsa-meets-earth/pypsa-earth → click **Fork** → rename the fork to `pypsa-libya` in your account settings.

3. Clone your fork and merge in the scaffold

```
git clone https://github.com/<you>/pypsa-libya.git
cd pypsa-libya
git remote add upstream https://github.com/pypsa-meets-earth/pypsa-earth.git

# copy the scaffold files into the cloned repo (don't overwrite upstream's own
config.yaml/Snakefile)
cp -r /path/to/pypsa-libya-scaffold/README.md .
cp -r /path/to/pypsa-libya-scaffold/REPO_STRUCTURE.md .
cp -r /path/to/pypsa-libya-scaffold/config.libya.yaml .
cp -r /path/to/pypsa-libya-scaffold/data_LY .
cp -r /path/to/pypsa-libya-scaffold/paper .
cat /path/to/pypsa-libya-scaffold/.gitignore >> .gitignore   # merge, don't overwrite upstream's
.gitignore
```

4. Commit and push

git add README.md REPO_STRUCTURE.md config.libya.yaml data_LY paper .gitignore
git commit -m "Add Libya scaffold: config, cost overrides, paper skeleton"
git push origin main

5. Set up the environment and do a first dry run

mamba env create -f envs/environment.yaml
conda activate pypsa-earth
snakemake --configfile config.yaml config.libya.yaml -j1 -n solve_all_networks

The `-n` is a dry run — it'll show you the full task DAG without downloading or computing anything, so you can sanity-check the config actually resolves before committing to a real run (which will pull weather cutouts and OSM data, easily tens of GB).

6. The real bottleneck: data Everything above is plumbing. The thing that actually determines whether PyPSA-Libya produces something meaningful is filling in `data_LY/costs/costs_updates.csv` with real figures, and — more importantly — finding *any* source for Libyan generation capacity by region and historical demand, since that's what PyPSA-Spain's whole improvement over the vanilla model was built on. If you can get even coarse GECOL annual reports or IEA/EIA country data, that's worth more than any config tweaking.

Want help next with drafting a data-collection checklist (what fields you'd need, and where such things are typically published for a country like Libya), or with getting the actual `snakemake` dry run working through any errors you hit?

==========

https://claude.ai/share/d866f430-bc30-440e-96c1-0451ebbf5494

Using config.libya.yaml and config.yaml

ECMWF Registration
https://cds.climate.copernicus.eu/profile
User: mhdabuella@gmail.com
PW: Mh..a1357+
url: https://cds.climate.copernicus.eu/api
key: 6ff8b423-dbc4-44be-92bb-5f71f5a50000

(pypsa-earth) C:\Users\Mhdella\Documents\pypsa-libya>snakemake --cores all solve_all_networks

Phew!!
We now have the results folder, finally!
what next?
—----------

That's a genuine milestone — congratulations, seriously. You just took a repo you'd never touched before, through cutout builds, OSM parsing, CDS auth, solver installs, and three separate real upstream bugs, all the way to:

Status: ok
Termination condition: optimal
Objective: 3.30e+10

A full, working, optimal solve for Libya. That's the hard part done — everything from here is refinement, not plumbing.

Here's what I'd do next, roughly in order:

1. Actually look at what it solved — sanity-check the result

Before anything else, open the solved network and see if the numbers make any sense at all. In your `pypsa-earth` conda env:

python

```
import pypsa
n = pypsa.Network("results/libya-test/networks/elec_s_8_ec_lcopt_Co2L-3h.nc")
print(n.generators.groupby("carrier").p_nom_opt.sum())
print(n.buses)
```

You're looking for red flags like: all capacity dumped onto one carrier, absurd values (thousands of GW), or only 1–2 buses actually having generators. Given the disconnected sub-network islands we saw earlier, don't be shocked if it looks lopsided — that's useful diagnostic information, not necessarily a new bug.

2. Commit this working state to git — lock in the progress

You've earned a checkpoint. Don't lose this working config to an accidental edit later.

```
cd C:\Users\Mhdella\Documents\pypsa-libya
git add config.yaml
```

git commit -m "Working end-to-end small test run: LY, 1 day, 8 clusters, HiGHS solve"
git push origin main

(If `git status` shows a huge pile of `resources/`, `networks/`, `cutouts/`, `results/` files wanting to be added too — **don't add those**, they should be gitignored per the scaffold from way back. If they're showing up as untracked, your `.gitignore` isn't catching them; tell me and we'll fix that before committing.)

3. Widen the test carefully, one variable at a time

Now that the pipeline works, resist the urge to jump straight to "the real thing" (full year, sector coupling, real CO2 target). Change one dimension, rerun, confirm it still works:

- Extend `snapshots` from 1 day to 1 week, then 1 month, then full year — watch memory/runtime scale
- Increase `clusters` from 8 toward something like 15–25
- Only after those are stable, revisit the CO2 constraint with a real assumption instead of the placeholder

4. Investigate the disconnected sub-network issue properly

This is the one open question actually worth understanding before treating results as meaningful — go back to why `distribute_clusters` needed to run at all (i.e., how many sub-networks did it find, and does that correspond to something real like Libya's south being weakly grid-connected, or is it an OSM data gap). Worth a `python` session digging into `n.buses.groupby(["country","sub_network"]).ngroups` type checks.

5. Circle back to the paper/README scaffold from earlier

Once you have a result you trust even a little, this is exactly the point to start filling in the `paper/paper.tex` skeleton and the README's limitations section — while everything's still fresh, not after you've moved on to bigger runs and forgotten which config produced what.

Want to start with #1 (actually looking at the solved network) right now, or do the git commit first while it's top of mind?

==========

https://claude.ai/share/d866f430-bc30-440e-96c1-0451ebbf5494

Using config.libya.yaml and config.yaml

ECMWF Registration
https://cds.climate.copernicus.eu/profile

User: mhdabuella@gmail.com

PW: Mh..a1357+

url: https://cds.climate.copernicus.eu/api

key: 6ff8b423-dbc4-44be-92bb-5f71f5a50000

(pypsa-earth) C:\Users\Mhdella\Documents\pypsa-libya>snakemake --cores all solve_all_networks

And then after I got the results folder.
I run %runfile C:/Users/Mhdella/Documents/pypsa-libya/results_v1.py --wdir
—----------

Good import pypsa n = pypsa.Network("results/libya-test/networks/elec_s_8_ec_lcopt_Co2L-3h.nc") print(n.generators.groupby("carrier").p_nom_opt.sum()) print(n.buses) gives Any observations, commetns and takeaways?

Python 3.10.20 | packaged by conda-forge | (main, Jun 11 2026, 03:28:51) [MSC v.1944 64 bit (AMD64)] Type "copyright", "credits" or "license" for more information. IPython 8.39.0 -- An enhanced Interactive Python. Type '?' for help. %runfile C:/Users/Mhdella/Documents/pypsa-libya/results_v1.py --wdir C:\Users\Mhdella\anaconda3\envs\pypsa-earth\lib\site-packages\google\api_core_python_version_support.py:254: FutureWarning: You are using a Python version (3.10.20) which Google will stop supporting in new releases of google.api_core once it reaches its end of life (2026-10-04). Please upgrade to the latest Python version, or at least Python 3.11, to continue receiving updates for google.api_core past that date. warnings.warn(message, FutureWarning) INFO:pypsa.io:Imported network elec_s_8_ec_lcopt_Co2L-3h.nc has buses, carriers, generators, global_constraints, links, loads, storage_units, stores carrier CCGT 14287.855400 load shedding 8610.162572 oil 6280.586964 onwind 4335.863758 solar 15445.754602 Name: p_nom_opt, dtype: float64 v_nom x y ... v_mag_pu_min v_mag_pu_max sub_network Bus ...
LY0 0 380.0 15.071779 31.558736 ... 0.0 inf
LY1 0 380.0 13.147500 32.730000 ... 0.0 inf
LY2 0 380.0 13.007400 32.617000 ... 0.0 inf
LY3 0 380.0 20.270900 31.926800 ... 0.0 inf
LY4 0 380.0 13.283500 30.428200 ... 0.0 inf
LY5 0 380.0 14.813400 27.322900 ... 0.0 inf
LY6 0 380.0 22.314700 32.793200 ... 0.0 inf
LY7 0 380.0 12.892300 32.055700 ... 0.0 inf
LY0 0 H2 1.0 15.071779 31.558736 ... 0.0 inf
LY1 0 H2 1.0 13.147500 32.730000 ... 0.0 inf
LY2 0 H2 1.0 13.007400 32.617000 ... 0.0 inf
LY3 0 H2 1.0 20.270900 31.926800 ... 0.0 inf
LY4 0 H2 1.0 13.283500 30.428200 ... 0.0 inf
LY5 0 H2 1.0 14.813400 27.322900 ... 0.0 inf

LY6 0 H2 1.0 22.314700 32.793200 ... 0.0 inf
LY7 0 H2 1.0 12.892300 32.055700 ... 0.0 inf
[16 rows x 17 columns]

—----------

This is genuinely useful output — here's what jumps out.

The single biggest red flag: `load shedding` at 8,610 MW

CCGT 14287.9
load shedding 8610.2 <-- this one
oil 6280.6
onwind 4335.9
solar 15445.8

"Load shedding" is PyPSA's way of representing *unmet demand* — it's modeled as an artificially expensive fake generator (per `config.default.yaml`, `load_shedding: 100` €/kWh = **100,000 €/MWh**, wildly above any real generation cost) so the solver only uses it as an absolute last resort. The fact that the optimizer chose to build 8.6 GW of it, despite already having ~40 GW of real generation capacity available (14,288 + 6,281 + 4,336 + 15,446), means **some part of the network structurally cannot be served by that generation** — not that Libya doesn't have enough total capacity, but that the capacity and the demand aren't reachable from each other.

This lines up exactly with the disconnected-sub-network issue we kept bumping into during clustering (`distribute_clusters` needing to run at all, the SCIP quadratic-solve step). It also connects to something in your solve log you may not have clocked:

No expandable lines found. Skipping iterative solving.

That means the transmission lines in this model are **fixed at whatever OSM happened to capture** — not something the solver is allowed to build or upgrade. If OSM's mapping of Libya's grid is sparse (which is plausible, and something we flagged early on as a real data-quality gap), some clustered buses may end up with weak or zero connections to the rest of the network, so their local demand has nowhere to draw power from except the fictitious load-shedding generator.

What I'd check next, to confirm this diagnosis rather than assume it:

```
print(n.lines[["bus0", "bus1", "s_nom", "s_nom_opt"]])
print(n.buses.groupby("sub_network").size())
```

```
# which buses are actually using load shedding, and how much
ls = n.generators[n.generators.carrier == "load shedding"]
print(ls[["bus", "p_nom_opt"]])
```

If you see multiple distinct `sub_network` values, or specific buses with large load-shedding capacity and few/no lines touching them, that confirms it's an islanding problem rather than a genuine capacity shortfall.

On the bus table itself:

8 electricity buses + 8 paired H2 buses = 16, matching your `clusters: [8]` setting correctly — clustering did what it was told. One loose end: the `sub_network` column printed **blank** for every bus, which is odd — worth explicitly checking with `print(n.buses.sub_network.unique())` since a blank display could just be a pandas truncation artifact, or it could mean something didn't get set. Worth ruling out before trusting the rest.

On the numbers' plausibility:

~40 GW of real generation capacity for Libya (real-world peak demand is roughly 6–7 GW today) is on the high side — though that's not necessarily wrong, since this is a single arbitrary day's weather/demand pattern being used to size year-round capacity (a known, expected distortion of the "1 day for testing" shortcut, not a new bug), and the `co2.limit` placeholder we set was deliberately non-binding, which lets the optimizer lean harder on cheap-but-intermittent renewables backed by lots of firm capacity. Once you extend to a full year, expect these absolute numbers to shift meaningfully — don't treat this run's capacity mix as a real answer yet, just as confirmation the mechanics work.

Bottom line takeaway: the pipeline is solving correctly and producing an internally consistent optimal solution — but that solution is telling you something true and important about your input data: Libya's OSM-derived transmission network likely has real connectivity gaps. That's valuable to know now, before you scale up to a full-year run and burn time/compute on a bigger version of the same underlying data problem.

==========

https://claude.ai/share/d866f430-bc30-440e-96c1-0451ebbf5494

Good First, based on the config.yaml file attached, is this total or complete network structure or just part of it?

Then, we also ran:

print(n.lines[["bus0", "bus1", "s_nom", "s_nom_opt"]])
print(n.buses.groupby("sub_network").size()) # which buses are actually using load shedding, and how much ls = n.generators[n.generators.carrier == "load shedding"] print(ls[["bus", "p_nom_opt"]])

Any observations, commetns and takeaways? [16 rows x 17 columns] %runfile C:/Users/Mhdella/Documents/pypsa-libya/results_v1.py --wdir INFO:pypsa.io:Imported network elec_s_8_ec_lcopt_Co2L-3h.nc has buses, carriers, generators, global_constraints, links, loads, storage_units, stores carrier CCGT 14287.855400 load shedding 8610.162572 oil 6280.586964 onwind 4335.863758 solar 15445.754602 Name: p_nom_opt, dtype: float64 v_nom x y ... v_mag_pu_min v_mag_pu_max sub_network Bus ...
LY0 0 380.0 15.071779 31.558736 ... 0.0 inf
LY1 0 380.0 13.147500 32.730000 ... 0.0 inf
LY2 0 380.0 13.007400 32.617000 ... 0.0 inf
LY3 0 380.0 20.270900 31.926800 ... 0.0 inf
LY4 0 380.0 13.283500 30.428200 ... 0.0 inf
LY5 0 380.0 14.813400 27.322900 ... 0.0 inf
LY6 0 380.0 22.314700 32.793200 ... 0.0 inf
LY7 0 380.0 12.892300 32.055700 ... 0.0 inf
LY0 0 H2 1.0 15.071779 31.558736 ... 0.0 inf
LY1 0 H2 1.0 13.147500 32.730000 ... 0.0 inf
LY2 0 H2 1.0 13.007400 32.617000 ... 0.0 inf
LY3 0 H2 1.0 20.270900 31.926800 ... 0.0 inf
LY4 0 H2 1.0 13.283500 30.428200 ... 0.0 inf
LY5 0 H2 1.0 14.813400 27.322900 ... 0.0 inf
LY6 0 H2 1.0 22.314700 32.793200 ... 0.0 inf
LY7 0 H2 1.0 12.892300 32.055700 ... 0.0 inf
[16 rows x 17 columns] Empty DataFrame Columns: [bus0, bus1, s_nom, s_nom_opt] Index: [] sub_network 16 dtype: int64 bus p_nom_opt Generator
LY0 0 load shedding LY0 0 8116.507731 LY1 0 load shedding LY1 0 137.414220 LY2 0 load shedding LY2 0 88.236675 LY3 0 load shedding LY3 0 41.279001 LY4 0 load shedding LY4 0 174.573682 LY5 0 load shedding LY5 0 20.003877 LY6 0 load shedding LY6 0 31.538338 LY7 0 load shedding LY7 0 0.609048 LY0 0 H2 load shedding LY0 0 H2 0.000000 LY1 0 H2 load shedding LY1 0 H2 0.000000 LY2 0 H2 load shedding LY2 0 H2 0.000000 LY3 0 H2 load shedding LY3 0 H2 0.000000 LY4 0 H2 load shedding LY4 0 H2 0.000000 LY5 0 H2 load shedding LY5 0 H2 0.000000 LY6 0 H2 load shedding LY6 0 H2 0.000000 LY7 0 H2 load shedding LY7 0 H2 0.000000

—----------

Good question, let's untangle both.

1. Is this the total/complete network, or just part of it?

Geographically, this config **does cover all of Libya** — `countries: ["LY"]` pulls OSM power infrastructure for the whole country, not a sub-region, and the demand/renewable data spans the whole geographic extent too. So it's not a partial slice of Libya's territory.

But "complete" in terms of *what got captured* is a different question — and your new output just answered it, unfortunately:

Empty DataFrame
Columns: [bus0, bus1, s_nom, s_nom_opt]
Index: []

There are zero transmission lines in this network. None at all. Not "some missing," literally zero rows. That's the real, now-confirmed root cause of the load shedding — it's not that Libya's grid has weak interconnections, it's that this particular solved network has **no grid at all connecting any of the 8 buses**. Each bus is a fully isolated island that can only use whatever generation was placed directly on it.

That also explains:

sub_network
16

All 16 buses (8 electricity + 8 H2) fall into a single blank-labeled group — not because they're connected, but because with zero lines, `determine_network_topology()` never had anything to group by, so nothing meaningful got computed.

And it explains the load-shedding table perfectly:

LY0 0 load shedding 8116.5 <- huge, one bus carrying almost all of it
LY1 0 load shedding 137.4
LY2 0 load shedding 88.2
...
LY7 0 load shedding 0.6 <- tiny, this bus is nearly self-sufficient

Every single electricity bus has *some* load shedding — consistent with each being isolated and only able to cover demand from whatever local generation the optimizer built right there. LY0 is the worst because it evidently has the most demand relative to whatever generation could physically be sited/built at that one bus alone.

2. Where did the lines actually disappear?

This is worth pinning down precisely rather than guessing, because we know from earlier logs that raw OSM extraction *did* find real line data:

Region LY (power=substation,line,cable,generator -> line): node rows=0, way rows=2355, total=2355

2,355 line features existed at the very first extraction step. Somewhere between there and your final solved network, all of them got dropped. Let's find exactly which pipeline stage did it:

```
import pypsa

for f in [
    "networks/libya-test/base.nc",
    "networks/libya-test/elec.nc",
    "networks/libya-test/elec_s.nc",
    "networks/libya-test/elec_s_8.nc",
    "networks/libya-test/elec_s_8_ec.nc",
]:
    n = pypsa.Network(f)
    print(f, "-> lines:", len(n.lines), " buses:", len(n.buses))
```

This checks line count at every stage: right after `base_network` (raw OSM→PyPSA conversion), after `add_electricity`, after `simplify_network`, after `cluster_network`, after `add_extra_components`. Whichever step shows the count drop to zero is the one to investigate — and given `base_network` is the very first PyPSA-format network built directly from OSM, if lines are already zero there, the bug is in `build_osm_network`/`base_network`'s voltage filtering (recall `osm.clean_osm_data.threshold_voltage: 51000` — if Libya's OSM line tags don't carry voltage info the way the script expects, everything could get filtered out as "unclassifiable, assume below threshold"). If lines are still present at `base.nc` but gone by `elec_s.nc`, the bug is in `simplify_network` instead.

Run that and paste back the output — that'll tell us exactly where to dig next rather than speculating further.

==========

https://share.gemini.google/eBgHHC6kI9Hi

Here is a breakdown of your questions covering Snakemake execution, PyPSA functionality, and power system optimization concepts.

Q1 & Q2: Understanding Snakemake Commands & Syntax

A) What is Snakemake doing?

Snakemake is a workflow management system. Think of it like a master project builder (similar to make or docker-compose). Instead of running dozens of Python scripts manually in sequence (e.g., download data $\rightarrow$ build cutouts $\rightarrow$ create grid $\rightarrow$ solve model), Snakemake looks at a master blueprint called a Snakefile.
When you ask Snakemake for a final target (like solve_all_networks), it looks at all existing files in your directory, determines which steps are missing or out of date, and automatically runs *only* the scripts needed to build the final output.

B) Breaking Down the Command Line From Root to End

```
Plaintext
(pypsa-earth) C:\Users\Mhdella\Documents\pypsa-libya>snakemake --configfile config.yaml
config.libya.yaml --cores all solve_all_networks
```

Segment	What It Means
(pypsa-earth)	**Conda/Virtual Environment:** Indicates your active Python environment containing PyPSA, Snakemake, and required packages.
C:\Users\Mhdella\Documents\pypsa-libya>	**Working Directory:** The root folder on your Windows machine where your Snakefile, configuration files, and project data reside.
snakemake	**Executable:** Invokes the Snakemake workflow manager engine.
--configfile config.yaml config.libya.yaml	**Configuration Files:** Passes settings to the workflow. config.yaml supplies default global settings, while config.libya.yaml overrides those settings specifically for Libya.

`--cores all`	**Parallel Processing:** Instructs Snakemake to utilize all available CPU cores on your machine to execute tasks faster.
`solve_all_networks`	**Target Rule:** The final goal/output rule defined in the `Snakefile`. Snakemake works backward from this rule to execute any preceding missing dependencies.

Comparing Command Variations

Difference 1: Passing Config Files vs. `--rerun-incomplete -call`

DOS

call

- **Command A** explicitly overrides configuration parameters by passing `config.yaml` and `config.libya.yaml`.
- **Command B**:
  - Omits explicit config file declarations (it defaults to using whatever config file path is hardcoded in the `Snakefile`, usually just `config.yaml`).
  - `--rerun-incomplete`: If a previous run crashed or was interrupted mid-way (e.g., leaving corrupted or half-written output files), Snakemake will re-run those incomplete tasks rather than assuming they finished successfully.
  - `-call` *(or* `-c all`*)*: Shorthand syntax telling Snakemake to use all CPU cores.

Difference 2: With vs. Without `--configfile` Flags

DOS

```
# Command 1
snakemake --configfile config.yaml config.libya.yaml --cores all solve_all_networks

# Command 2
snakemake --cores all solve_all_networks
```

- **Command 1** merges two configuration files. `config.libya.yaml` overrides matching keys in `config.yaml`. This ensures settings like custom Libyan boundaries, coordinates, and cutouts are applied.
- **Command 2** skips the user-defined overrides and falls back strictly to the default configuration defined inside the `Snakefile` (which usually only loads `config.yaml`). If your

country-specific parameters are stored in config.libya.yaml, Command 2 will **not** load them.

Q3: Power System Optimization Concepts

A & B) Is it Economic Dispatch or Optimal Power Flow (OPF)?

It is an **Optimal Power Flow (OPF)**—specifically, a **Linearized AC / DC-Optimal Power Flow (DC-OPF)** combined with **Capacity Expansion Planning (CEP)**.
Here is how it compares to standard Economic Dispatch:

- **Economic Dispatch:** Ignores grid topology and line constraints. It simply matches generation to load at the lowest operational cost.
- **DC-OPF (What PyPSA uses):** Considers:
    - Generator operational costs (EUR/MWh) and renewable availability profiles ($p_{\text{max,pu}}$).
    - **Network constraints:** Power transfers must respect Kirchhoff's Laws (Voltage angles and Line Reactances X) and maximum transmission line thermal limits (S_{nom}).
    - **Capacity Expansion (Co-optimization):** PyPSA not only dispatches existing generators, but can also optimize where to *build new* generation assets and transmission lines to minimize total system cost.

(Note: PyPSA models power flows using linear DC approximations rather than full non-linear AC equations to solve large multi-period energy transition models efficiently).

C) What Does the Solver Output Mean?

```
Plaintext
Status: ok
Termination condition: optimal
Objective: 3.30e+10
```

- Status: ok: The mathematical solver (e.g., Highs, Gurobi, or CBC) ran without internal software errors or numerical issues.
- Termination condition: optimal: The solver found the **globally optimal solution**. This means no other combination of generation dispatch and capacity investments exists that satisfies all load and network constraints at a lower cost.
- Objective: 3.30e+10: The total system cost minimized by the solver, expressed in scientific notation ($3.30 \times 10^{10} = 33,000,000,000$).
    - In PyPSA, this typically represents the total annualized system cost in **Euros (€33 Billion)**, including both fixed annualized investment costs (CAPEX) for power plants/lines and variable operational costs (OPEX + fuel).

==========

https://share.google/aimode/ZEMcbWsvSCUg1VfGF

PyPSA-earth is an incredible tool that has quickly become a cornerstone of global open-source energy planning. It is a massive breakthrough because it allows anyone to model a sustainable energy transition for any country or region on Earth for free. [1, 2, 3, 4]

---

1. What Is It?

PyPSA-earth is a **global, open-source, multi-energy system optimization model** with high spatial (geographic) and temporal (hourly) resolution. [2, 4]

-
- **The Core Framework:** It is built on top of **PyPSA** (Python for Power System Analysis), which was originally developed by researchers at the Technical University of Berlin. [2, 5, 6, 7, 8]
- **The "Earth" Blueprint:** PyPSA-earth takes that codebase and connects it to global public data pipelines. It automatically extracts power grid lines from OpenStreetMap, gathers satellite weather data for wind and solar potentials, and pulls electricity demand profiles to create a ready-to-run simulation of any country. [2, 9, 10, 11, 12, 13]
- **Cross-Sector Capabilities:** It does not just model electricity. It is "sector-coupled," meaning it can optimize interactions between electricity, heating, transport, industry, hydrogen, and more. [1, 12]
-

---

2. When and by Whom Was It Established?

-
- **When:** The *PyPSA meets Earth* initiative officially launched in **late 2021 / early 2022**. It grew out of a desire to expand the highly successful European grid model (*PyPSA-Eur*) to the rest of the world. [2, 4, 14, 15]
- **By Whom:** It was founded by an international, independent collective of energy researchers, data scientists, and engineers. The initiative is led by prominent academic and research figures like **Davide Fioriti** and supported by global institutions, universities, and open-source contributors. The community thrives collaboratively via its GitHub Organization and an active community Discord channel. [4, 5, 16, 17]
-

---

3. Why Was It Established? (The Problem It Solves)

Before PyPSA-earth, energy planning was hindered by major institutional barriers: [2, 18]

-
- **The Problem with "Black Box" Models:** Most national governments and industries use massive, proprietary, closed-source software to map out their future energy

infrastructure. These systems cost tens of thousands of dollars to license, and their inner logic is hidden. This makes it impossible for public citizens, independent scientists, or cash-strapped agencies to audit their findings or trust their policy decisions.

- **Geographical and Data Inequality:** Richer nations (like the US and EU) have always had high-resolution, open energy models. However, developing regions—such as parts of Africa, Latin America, and Asia—were frequently left behind because they lacked aggregated, highly resolved transmission and resource data.
- **The Mission:** The founders established the *PyPSA meets Earth* initiative to democratize energy engineering. They wanted to offer **transparent analysis for an inclusive and sustainable energy future**. By creating a tool that can instantly generate a detailed, validated grid infrastructure model from scratch for *any* nation (like your current work with Libya), they leveled the global playing field. [1, 2, 4, 5, 11, 12, 17]
-

---

4. How It Helps Your Simulation

Because it is open and data-driven, you can use PyPSA-earth to test real-world scenarios for Libya's energy transition: [1, 19, 20]

- Calculate the absolute cheapest mix of solar and wind generation required to meet Libya's peak demands.
- Discover exactly where bottlenecks will occur in Libya's transmission corridors as old gas plants retire.
- Model precise net-zero carbon targets to see how much energy storage or battery capacity Libya would need to sustain itself through the night. [1, 5, 10, 11, 12, 19]

To help guide your deep dive into the platform, let me know if you would like to:

-
- Explore the **main data sources** (like OpenStreetMap or global weather databases) that PyPSA-earth is pulling from to generate your Libya network maps?
- Walk through an explanation of how a **"Capacity Expansion Study"** works to find out how many solar panels or lines to build?
- Learn how to **connect with the global community** on Discord if you want to contribute regional expertise on Libya's power system? [1, 2, 5, 9, 11, 12]

==========

https://share.gemini.google/MMgydENq1P0i

With --rerun-incomplete:

> Snakemake checks its memory/history. If it sees that a task stopped or crashed halfway during a previous attempt, it says, "This file was never finished properly. Let me delete it and start that specific task fresh."
>
> Sounds like a powerful command, I know that snakemake was established in 2012 for bioinformatics (genome/DNA research), when rerun-incomplete was introduced in the same release of snakemake that year or it comes later? When? Which release has it?
>
> ---

The `--rerun-incomplete` (or `--ri`) flag was **not** part of the original 2012 release of Snakemake. It was introduced later as workflow complexities grew and researchers started running massive parallel jobs on HPC clusters where premature job crashes were common.

Timeline & Evolution

1. The Early Years (2012 – 2014)

- **2012:** Johannes Köster published the initial version of Snakemake (Version 1.0).
- **How incomplete files were handled initially:** Back then, Snakemake relied almost purely on **file existence** and **file modification timestamps** (`mtime`). If a job was killed mid-execution, a half-written file left on the disk would fool Snakemake into thinking the task was finished. Users had to manually find and delete corrupted files using custom cleanup bash scripts.

2. Introduction of Incomplete Tracking (circa 2015–2016)

- Around **Snakemake v3.x – v4.x**, Snakemake added internal metadata tracking (stored inside the hidden `.snakemake/` directory in your project).
- Whenever a task starts, Snakemake marks its target files as **"incomplete"** in its local database. Only when the script/rule exits with a successful return code (code `0`) does Snakemake remove that "incomplete" tag.
- To complement this system, Snakemake introduced the `--rerun-incomplete` (`--ri`) flag. This allowed users to tell Snakemake: *"If you see any file marked as incomplete in the* `.snakemake` *metadata directory, delete it and run the step again."*

3. Modern Era (2022+ / Snakemake 7.8+)

- In **Snakemake 7.8** (released in 2022), the execution model underwent a major overhaul to move away from pure timestamp checks toward full **provenance tracking** (tracking changes in code, parameters, software environments, and file completion status).
- In modern Snakemake versions (v7, v8, and current v9), tracking incomplete files is a standard core safety feature, and using `--rerun-incomplete` is heavily recommended across large multi-rule workflows like PyPSA-Earth.

Summary

- **Was it in 2012 (v1.0)?** No.
- **When did it arrive?** It was introduced around **2015–2016** (during the **Snakemake 3.x / 4.x** cycle) when internal metadata tracking was built into `.snakemake/`.

> ==========

https://share.gemini.google/4RINNZ2mm4dV

Good

For this kind of projects like Pypsa-earth

Which is better to use: mamba vs conda? and Why?

—----------

For large, complex scientific modeling projects like **PyPSA-Earth**, mamba **is strongly recommended over standard** conda.
The PyPSA-Earth team explicitly recommends using mamba in their official setup documentation.

Why mamba is Better than conda for PyPSA-Earth

1. Faster Environment Creation

PyPSA-Earth relies on a massive stack of complex scientific libraries (such as xarray, geopandas, rasterio, pypsa, snakemake, atlite, and various mathematical solvers like highs or cbc).

- **Conda:** Can take **30 to 60+ minutes** just to figure out how to assemble all these packages together without conflicts. It frequently gets stuck indefinitely on *"Solving environment..."*.
- **Mamba:** Solves the same environment in **2 to 5 minutes**.

2. C++ Core Solver

- **Conda** uses a dependency solver written in Python, which is single-threaded and struggles with complex dependency trees.
- **Mamba** uses a C++ engine (libsolv) with parallel processing, which resolves complex library dependencies exponentially faster and uses significantly less RAM.

3. Better Error Messages

If there is a conflict in your environment.yaml file (for example, if two packages require incompatible versions of numpy), conda gives long, unhelpful walls of text. mamba clearly tells you exactly which two packages are fighting each other and why.

Comparison Summary

Feature	conda	mamba
Solving Engine	Python (Single-threaded)	C++ (libsolv, Multi-threaded)

Setup Time for PyPSA	Very Slow (30-60+ mins)	Fast (2-5 mins)
Memory usage	High (can run out of RAM)	Optimized
PyPSA Documentation Choice	Alternative / Backup	**Recommended Primary Tool**

How to Use Mamba for PyPSA-Earth

mamba is designed as a drop-in replacement for conda. All syntax is identical—you simply replace the word conda with mamba.

DOS

```
# Create the environment (Recommended method)

mamba env create -f envs/environment.yaml

# Note: Always activate using conda (mamba uses the conda environment manager under the hood)

conda activate pypsa-earth
```

==========

https://share.gemini.google/Qmo1IIBUpWWi

The process of open-source mapping and energy system modeling using PyPSA-Earth involves specific source data parameters, code configurations, and structural requirements.

1. OpenStreetMap (OSM) Recency for Libya

OpenStreetMap (OSM) data is not inherently pre-updated to 2026; its content depends on when the data extract is downloaded.

- **Extract Metadata:** Based on the binary file headers provided, the extracted OSM dataset is sourced from Geofabrik (https://download.geofabrik.de/africa/libya-updates) using Osmium tools.

- **Update Frequency:** Geofabrik updates its regional sub-extracts daily. When Snakemake executes the data retrieval step (`retrieve_osm_data` or `build_shapes`), it pulls the most recent OSM snapshot available at the time of execution.
- **Community Activity:** Open-source map updates rely entirely on volunteer mappers, satellite imagery availability, and regional contributions.

2. Sufficiency of OSM Data for Modeling Libya's Grid in 2026

While OSM provides a good spatial baseline, **OSM data alone is generally not sufficient to build a fully accurate power system model for Libya.** Additional data validation and enrichment are required for the following reasons:

Strengths of OSM Data

- **Geospatial Layout:** Excellent for mapping existing physical corridors, high-voltage transmission lines, and major substation locations.
- **Geographical Features:** Useful for identifying administrative boundaries and land-use constraints.

Key Limitations & Missing Parameters

1. **Technical Specifications:** OSM rarely includes technical power-flow parameters required by PyPSA (e.g., line impedance r and x, thermal limits s_{nom}, or specific transformer parameters). PyPSA-Earth uses heuristic rules to estimate missing electrical values based on voltage levels.
2. **Post-2020 Infrastructure Changes:** Significant grid damage, planned expansions, or recent reconstruction projects in Libya may not yet be digitized by OSM contributors.
3. **Power Plants & Generators:** Power plant operational capacities, fuel types, heat rates, and dynamic startup/shutdown costs are often incomplete or inaccurate in standard OSM tags.
4. **Demand & Load Profiles:** OSM provides physical geometry, not electrical demand. PyPSA-Earth relies on supplementary datasets (e.g., WorldPop, nightlight satellite data, or historical ENTSO-E/GEGIS style load curves) to distribute electrical demand across regions.

Recommendation: Combine OSM data with secondary domain sources, such as reports from the **General Electricity Company of Libya (GECOL)**, international energy databases (IEA, GEM - Global Energy Monitor), or satellite imagery verification.

3. Simplified Explanation of the PyPSA-Earth `Snakefile`

In PyPSA-Earth, `Snakefile` serves as the central workflow orchestration script managed by **Snakemake**. It defines the dependency graph, data processing steps, and execution pipeline needed to build and solve the electricity network model.

Main Workflow Pipeline

Raw Data / OSM Extracts]



[Data Processing Rules] ──► Clean topology, aggregate shapes, cut regions

|

▼

[Network Building Rules] ──► Estimate parameters, add generators & load

|

▼

[Optimization] ──► Solve linear optimal power flow (PyPSA / Linopy)

|

▼

[Plotting & Output Rules] ──► Export maps, CSVs, and capacity expansion results

Core Structure and Code Sections

A PyPSA-Earth Snakefile is structured into several functional rule sections:

1. Configuration and Setup

Loads global configuration options from config.yaml (such as regional scope, scenario target years, line expansion limits, and resolution parameters).

Python

```
configfile: "config/config.yaml"

# Sets up global variables, python libraries, and target outputs
```

2. Target Rule (rule all)

Acts as the entry point. It lists all final output files (e.g., solved network files, plot images, summary CSVs) that Snakemake must generate.

Python

```
rule all:
    input:
        "results/networks/elec_s_cop26.nc"
        "results/plots/map.png"
```

3. Data Extraction Rules (retrieve_*)

Downloads external datasets, such as Geofabrik OSM files, land-cover constraints, and load profiles.

```Python
rule retrieve_osm_data:
    output:
        "data/osm/libya-latest.osm.pbf"
    shell:
        "wget -O {output} https://download.geofabrik.de/africa/libya-latest.osm.pbf"
```

4. Topology and Network Assembly (`build_network` / `base_network`)

Converts raw geographic shapes and OSM power geometries into a PyPSA network object (`pypsa.Network`), mapping lines, buses, and transformers.

```Python
rule build_base_network:
    input:
        osm="data/osm/libya-latest.osm.pbf"
    output:
        network="resources/networks/base.nc"
    script:
        "scripts/build_base_network.py"
```

5. Optimization Rules (`solve_network`)

Calls solvers (such as Gurobi, Cbc, or GLPK) to perform Linear Optimal Power Flow (LOPF) or Capacity Expansion Planning (CEP).

```Python
rule solve_network:
    input:
        "resources/networks/elec_s.nc"
    output:
```

"results/networks/elec_s_solved.nc"

script:

"scripts/solve_network.py"

==========

https://claude.ai/share/d866f430-bc30-440e-96c1-0451ebbf5494

Good idea — visualizing this will make the "zero transmission lines" problem immediately obvious, and give you something concrete for the paper/README later. Let me write you a script for both parts.**How to use it:**

1. Download this file, save it directly into `C:\Users\Mhdella\Documents\pypsa-libya\` (same folder as `config.yaml`, `Snakefile`, etc.)

2. In your cmd window (make sure you're in the `pypsa-earth` conda environment and in that folder):

cd C:\Users\Mhdella\Documents\pypsa-libya

python visualize_libya.py

3. It'll create a new `viz_outputs\` folder with 7 PNG files:
   - `network_topology_base.png` through `network_topology_elec_s_8_ec.png` (5 images, one per pipeline stage) — **this is the important diagnostic set**, since it'll visually show you exactly which stage the lines vanish at
   - `solved_capacity_by_carrier.png` — the bar chart from our earlier numbers, now as a labeled chart
   - `solved_capacity_map.png` — geographic map of Libya with bus markers sized by capacity, colored by dominant carrier (solar/wind/CCGT/oil/load-shedding), with any transmission lines drawn as black connecting lines (expect this to show *no* connecting lines currently, visually confirming what we found)

What to look for once it runs:

- In the 5 topology images, watch the printed `buses=X lines=Y` line in your terminal for each stage. The moment `lines` drops to `0`, that's your answer to the earlier open question about where the lines disappeared.
- In `solved_capacity_map.png`, you should see 8 isolated colored dots scattered across Libya with no lines between them — a very visual, immediate confirmation of the islanding problem, worth keeping for the README's "known limitations" section as evidence, even before it's fixed.

One heads-up: `n.plot()` can occasionally throw errors if certain optional plotting dependencies (like `cartopy`) aren't installed — I've built in a fallback to plain matplotlib scatter/line plotting so the script won't just crash if that happens, but the maps will look more basic (no coastlines/borders) in that case. If you want proper geographic basemaps later (coastline of Libya, neighboring countries, etc.), that's a further step involving `cartopy` or `geopandas` + the `country_shapes.geojson` file already sitting in your `resources/libya-test/shapes/` folder — worth doing once we've actually fixed the missing-lines issue, so you're not polishing visuals for a broken network.

Run it and paste back the terminal output (the buses/lines counts per stage) plus let me know if any images fail to generate.